ポートフォリオ

末廣 航一

目次

01 自己紹介 p3

02 就職作品紹介 p4～

03 チーム制作作品紹介 p8～

04 その他過去作品紹介 p13～

プロフィール



日本工学院専門学校ゲームクリエイター科4年制
プログラマーコース2028年卒業予定

すえひろ こういち
末廣 航一

使用言語・ツール



Gravity Shooter

ジャンル：奥スクロール型 重力シューティング

制作時期：2026年4月～（制作途中）

開発人数：1人（個人制作）

開発環境・API

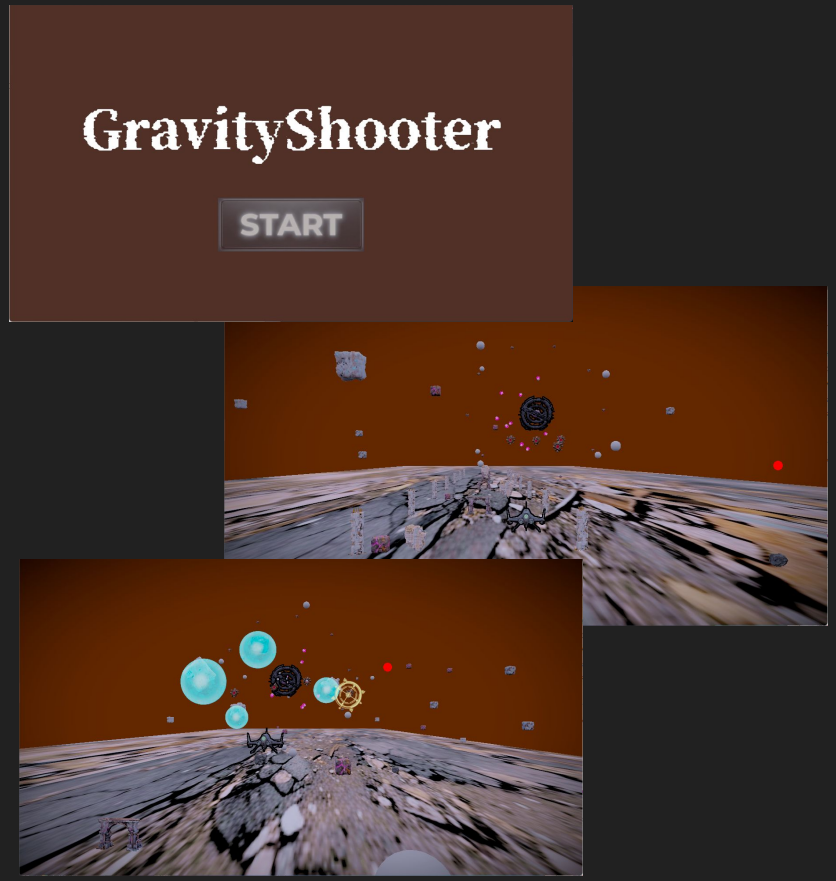
C++20 / Visual Studio 2026 / DirectX 12(描画) / XAudio2(サウンド) / Windows SDK 10.0.26100

外部ライブラリ

Assimp（3Dモデル読み込み）, DirectXTex（テクスチャ・画像読み込み）, nlohmann json（JSONデータのパース・保存）, Dear ImGui（リアルタイムデバッグUI）

【 担当箇所 】

- プログラム全般（企画・設計・実装）
  ※効果音・BGMと、一部3Dモデル・UI素材はフリーおよびAI生成素材を使用

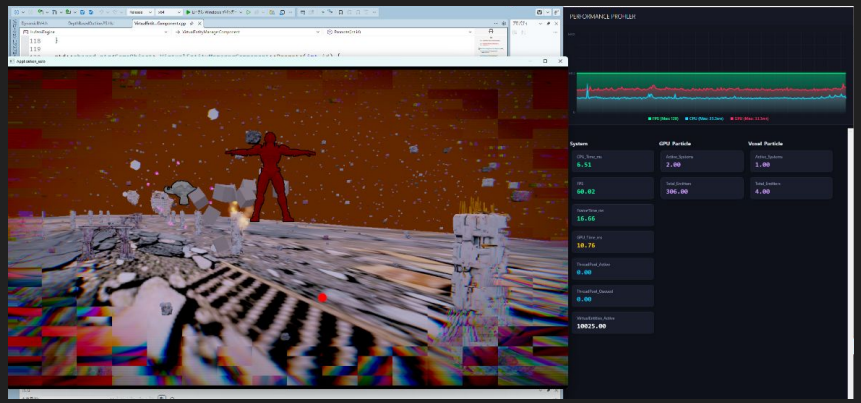


1万個を超えるガレキの描画と「騙し」の物理演算

【目的と課題】
「周囲の無数のガレキを引き寄せて敵に放つ」爽快感を実現したかったが、万単位のオブジェクトの物理演算を真面目に行うと CPUの限界を超えてしまう問題があった。

【解決策（実装）】
・賢い騙し（Fake Physics）：ガレキの引き寄せや浮遊中は重力演算を完全に切り、sin/cos 等を用いた軽量なローカルアニメーションのみで「重力による浮遊感」を表現した。
・厳格なメモリ管理： ゲームループ中における動的なメモリ確保（new/delete）を禁止。敵や弾だけでなく、各種ガレキも Prefab ごとに Object Pool を事前生成し、再利用する機構を構築した。

【定量的な成果】
画面上に1万個のガレキが存在しても処理落ちしない極限のパフォーマンスと、重力アクションの「手触りの良さ」を両立した。



ハードコードを排除したデータ駆動設計と　Dynamic BVH

【目的と課題】
マップごとの敵の出現位置や大量の背景をコードに直書きすると保守性が下がる。また、万単位のオブジェクトによる衝突判定の負荷が課題だった。

【解決策（実装）】
・Data-Driven構築：JSON/CSVでウェーブや環境を外部定義し、EnvironmentManager でプーリングとバッチ描画（インスタンシング）を自動化。
・Dynamic BVHの独自実装：衝突判定を $O(N \log N)$ にする動的AABBツリーを実装。さらにツリーのノード管理をポインタではなく **「インデックスベースの配列プール」**で実装し、キャッシュミスによる速度低下を完全に排除した。

【定量的な成果】
プログラマ視点での高い保守性とシーンファイルの肥大化防止、そして「大量のガレキが密集しても FPSが落ちない」圧倒的な実行速度（ CPUバウンドの解消）を同時に達成した。

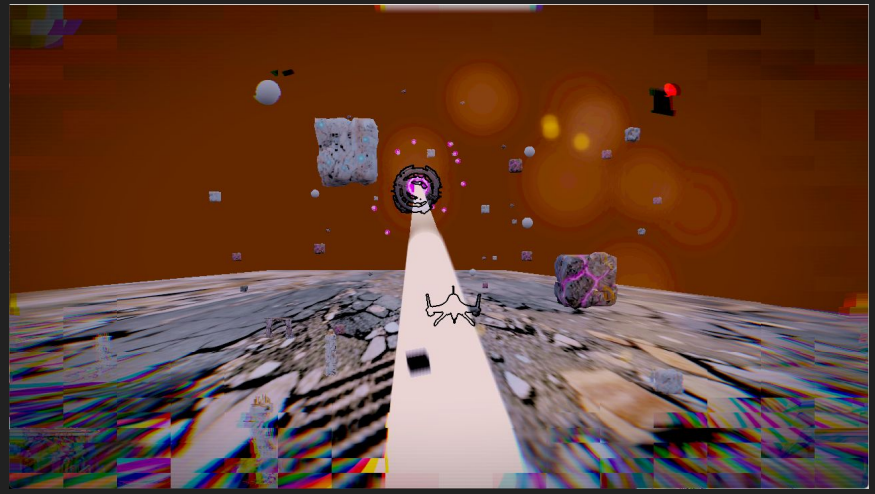
```
46      /// @brief ノードIDを指定してツリーから削除する
47      void Remove(int32_t nodeId);
48
49      /// @brief コライダーが移動した際にAABBを更新する（位置が大きく変わった場合は再挿入される）
50      void Update(int32_t nodeId, const AABB& newAABB);
51
52      /// @brief ツリー内の交差している全ペアを抽出する (Broad-Phase)
53      /// @param[out] outPairs 交差しているコライダーのペアリスト
54      void GetPotentialCollisionPairs(std::vector<std::pair<ColliderComponent*, ColliderComponent*>>& outPairs) co
55
56      /// @brief 指定したAABBと交差するコライダーを抽出する
57      void Query(const AABB& testAabb, std::vector<ColliderComponent*>& outColliders) const;
58
59      /// @brief レイキャスト走査を行い、ヒットしたコライダーを抽出する
60      void RaycastQuery(const Ray& ray, float maxDistance, std::vector<ColliderComponent*>& outHits) const;
61
62      /// @brief ツリーを空にする
63      void Clear();
64
65  private:
66      std::vector<BVHNode> nodes_;
67      int32_t rootIndex_ = -1;
68      int32_t freeListFirst_ = -1;
69
70      // --- 内部ヘルパー関数 ---
71
72      /// @brief 新しいノードを確保（またはフリーリストから再利用）する
73      int32_t AllocateNode();
74
75      /// @brief ノードをフリーリストに返却する
76      void FreeNode(int32_t nodeId);
77
78      /// @brief 2つのAABBを結合した新しいAABBを返す
79      AABB MergeAABB(const AABB& a, const AABB& b) const;
80
81      /// @brief AABBの表面積（Surface Area）を計算する
82      float SurfaceArea(const AABB& aabb) const;
83
84      /// @brief 葉ノードをツリーに挿入する
85      void InsertLeaf(int32_t leafIndex);
86
```

ビジュアル表現とテクニカルアート

【目的と課題】
ダークで退廃的なサイバーパンク風の世界観を表現するため、単なる3Dモデルではなく、手描きコミックのようなリッチな線画表現が必要だった。

【解決策（実装）】
・「深度・法線・輝度」の 3つのG-Bufferによる全体エッジ抽出に加え、マスクバッファを用いた「オブジェクトごとの個別アウトライン」を統合するマルチレイヤー・シェーダーを独自実装。
・線の太さ等のパラメータは定数バッファ化し、デバッグ UIから動的調整可能なデータ駆動設計を採用して表現の幅を拡大。 ・加えて、ノーマルマップでオフスクリーンバッファを歪ませる屈折シェーダーにより、ブラックホールのような重力場（空間の歪み）を表現した。

【定量的な成果】
自作エンジンだからこそ実現できる G-Bufferへの直接アクセスを活かし、ゲームのコンセプトに合致した説得力のあるビジュアルを確立した。



七転び八転び

ジャンル：3Dアクションゲーム

制作時期：2026年2月～5月（約4ヶ月）

開発人数：4人（プログラマーのみ）

開発環境・API

C++20 / Visual Studio 2026

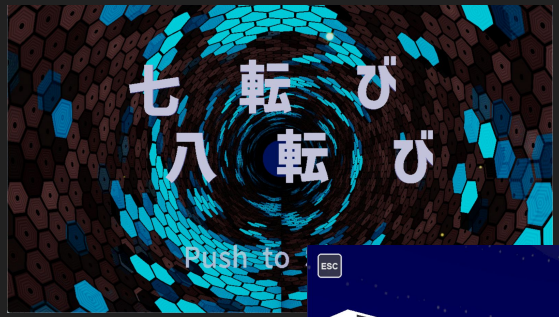
DirectX 12（描画）/ XAudio2（サウンド）/ Windows SDK 10.0.26100

外部ライブラリ

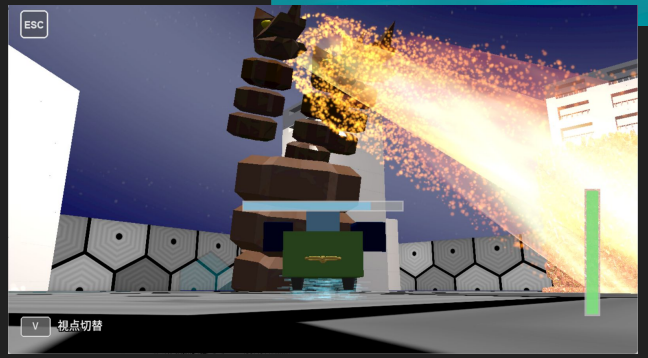
Assimp（3Dモデル読み込み）, DirectXTex（テクスチャ・画像読み込み）, nlohmann json（JSONデータのパース・保存）, Dear ImGui（リアルタイムデバッグUI）

【 担当箇所 】

- 自作ゲームエンジン（非同期ローダー・マルチバッファリング・シーン制御・リソース管理）
- 各種シーン実装（タイトル・クリア・ゲームオーバー）
- 描画・演出全般（独自HLSLシェーダー・Compute Shader等）
- チーム開発支援（ImGui連携デバッグUI・汎用APIの提供・ドキュメント整備）







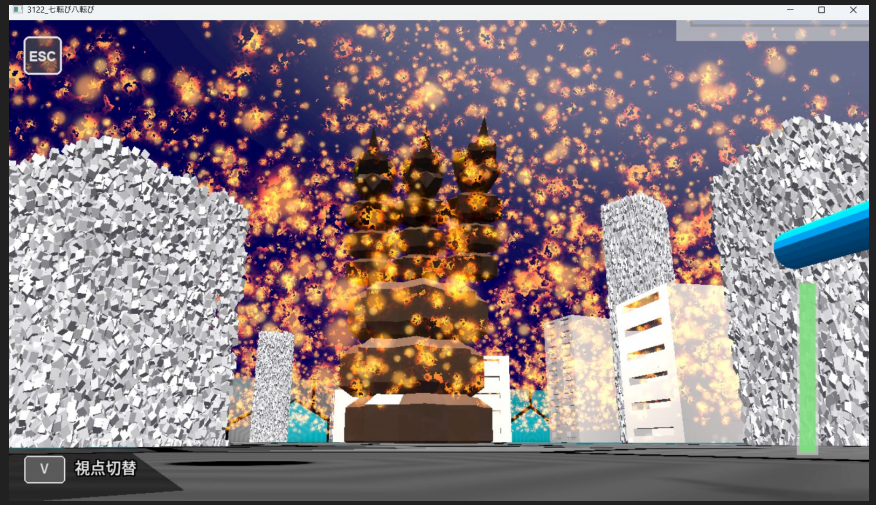
数百万のボクセル破壊を 60FPSで処理する最適化

【課題】
建物を壊す爽快感を追求した結果、数十万の破片（ボクセル）の物理演算がCPUの処理限界（16.6ms）を超過し、極端な処理落ちが発生。

【解決策（実装）】
ゲーム進行に必要な当たり判定（OBB等）はCPUに残し、視覚的な「大量の破片の物理演算」のみを **Compute Shader（GPU）へ完全にオフロード** する描画特化の設計へ移行。

【定量的な成果】

CPU側のディスパッチ負荷を 平均0.066ms に抑えることに成功。破壊の瞬間に1〜2フレームのみ30FPS相当に落ちるものの、即座に60FPSへ復帰し、ゲームのテンポを維持しました。



スレッドプールによる独自の非同期ローダー構築

【課題】

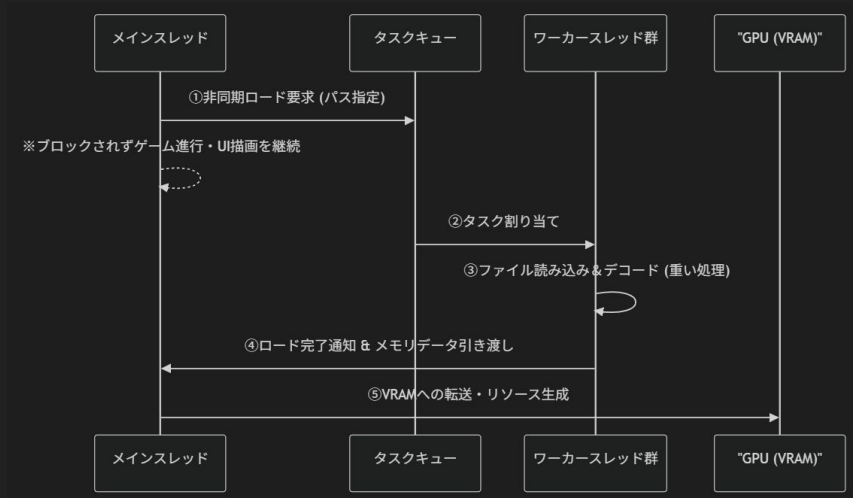
高解像度テクスチャのロードがメインスレッドを占有し、アクションゲームで最も重要な「テンポ」や「没入感」を削ぐ数秒間のフリーズ（スパイク）が発生。

【解決策（実装）】

OS依存の std::async ではなく、ゲーム進行を絶対に阻害しないよう、自前でワーカースレッド数と待機状態を完全制御できるスレッドプールを std::mutex を用いてスクラッチ実装。

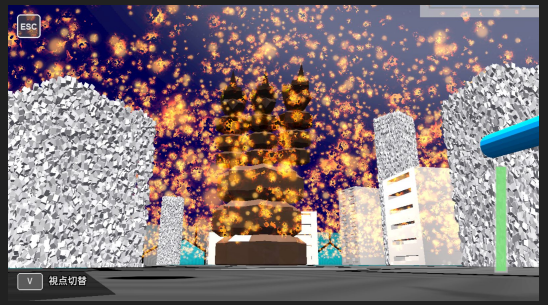
【定量的な成果】

ロード中の最大スパイクを 約4100ms から 約51ms に激減。ローディング画面のアニメーションを一切阻害しないシームレスな UXを実現しました。



アプリ終了時：全ワーカースレッドの待機状態を解除し、確実に join() して破棄することでメモリリークを防止。
シーン破棄時の対応：ロード完了前にシーンが遷移・破棄された場合でも、取得したリソースは次回のキャッシュとして保持（または安全に解放）する仕組みを構築。

ボリュームレンダリングによる立体的爆煙表現



【目的と要件】

ボスの大迫力な攻撃を表現するため、従来の板ポリゴンでは不可能な「煙の厚み」や「内部で光が散乱する自己遮蔽」を描画したいと考えた。

【実装と最適化】

GPU負荷が上がるリスクを承知の上で、HLSLによるボリュームレンダリング（レイマーチング）をスクラッチで実装。負荷を極限まで下げるため、煙を覆う「境界球（Bounding Sphere）」との交差判定を数学的に事前計算し、レイを飛ばすサンプリング区間を最小限に絞り込む最適化を施した。

【成果】

3D空間ノイズ（FBM等）を用いた濃密な爆煙の質感を、許容フレームレート（処理時間内）に収まるパフォーマンスで実現した。

```
// --- ボリュームレンダリング(レイマーチング)の最適化 ---

// 1. 煙を覆う「境界球(Bounding Sphere)」との交差判定を数学的に事前計算
float t0, t1; // レイが入る距離(t0)と出る距離(t1)
if (!IntersectSphere(rayOrigin, rayDir, explosionCenter, explosionRadius, t0, t1)) {
    discard; // 球をかすりもしないピクセルは即座に描画スキップ（負荷削減）
}

// カメラが煙の中にある場合のケア
t0 = max(t0, 0.0);

// レイが煙(球体)の中を進む実質の距離
float tMax = t1 - t0;
if (tMax <= 0.0) discard;

// 2. レイマーチング開始（サンプリング区間を球体内部のみに極限まで絞り込む）
const int STEPS = 32; // サンプリング回数
float stepSize = tMax / float(STEPS); // 1歩の歩幅
float tCurrent = t0;

for (int i = 0; i < STEPS; i++) {
    float3 pos = rayOrigin + rayDir * tCurrent;

    // 3. 3D空間ノイズ(FBM)による濃密な煙のサンプリング
    float noiseVal = fBm(pos * noiseScale - dir * time);
    float density = CalculateDensity(noiseVal, pos);

    if (density > 0.01) {
        // 色と不透明度の合成（Pre-multiplied Alpha）
        float alpha = density * 0.25;
        accumColor.rgb += GetFireColor(noiseVal) * alpha * (1.0 - accumColor.a);
        accumColor.a += alpha * (1.0 - accumColor.a);

        // 4. 完全に不透明になったら、それより奥の計算を打ち切る（早期リターン）
        if (accumColor.a > 0.99) break;
    }

    tCurrent += stepSize;
}
```

ロジック担当視点の API設計とツール提供による開発支援

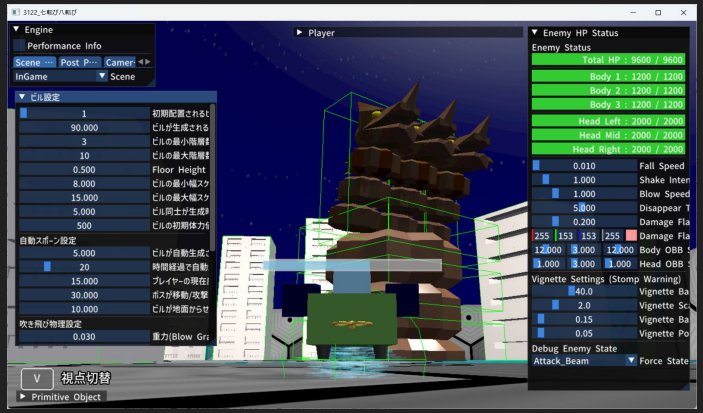
【課題】
高度なシェーダーを作っても、他のメンバーが扱えなければ組み込まれず、またパラメータ調整のたびに「再ビルド」が発生し開発のボトルネックになっていました。

【解決策と成果】
1. Facadeパターンの直感的なAPI設計：複雑なGPU処理を隠蔽し、1行呼ぶだけでリッチな演出を出せるAPI（Engine::SpawnEffect 等）を提供。

2. ImGuiを用いたリアルタイムデバッグ：ゲームを起動したままエフェクトの色や速度を調整できる環境を構築し、無駄な待ち時間を排除。

3. 知識の資産化：2万文字規模のマニュアルを執筆。「作って、試して、共有する」イテレーションを高速化し、エンジニアとしてチームの開発基盤を支えました。





纏当て

ジャンル：3Dボスバトルアクション

制作時期：2025年11月～12月（約1ヶ月）

開発人数：4人（プログラマーのみ）

開発環境・API

C++20 / Visual Studio 2026 / DirectX 12(描画) / XAudio2(サウンド) / Windows SDK 10.0.26100

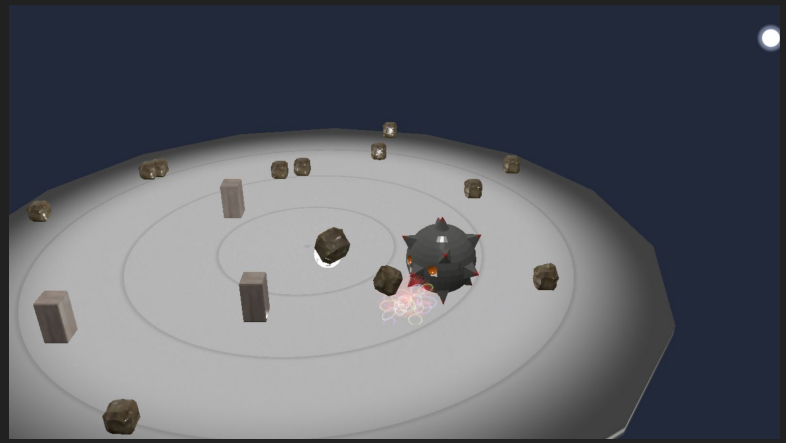
外部ライブラリ

Assimp（3Dモデル読み込み), DirectXTex（テクスチャ・画像読み込み), Dear ImGui（リアルタイムデバッグUI）

【 担当箇所 】

- 自作ゲームエンジンの開発全般（DirectX 12 描画パイプラインの構築、パーティクルシステム、デバッグUI環境の提供、シーン管理基盤など）
- シェーダー実装（描画用HLSL）
- アセット作成(自機、敵、纏うブロック)

※敵AIやプレイヤー制御などのゲームロジックは他メンバーが担当。自身はチームメンバーがゲーム開発をしやすい基盤作りに注力しました。





血管壊回

ジャンル：見下ろし型 疑似3Dアクション

制作時期：2026年1月～2月（約1ヶ月）

開発人数：3人（プログラマーのみ）

開発環境・API

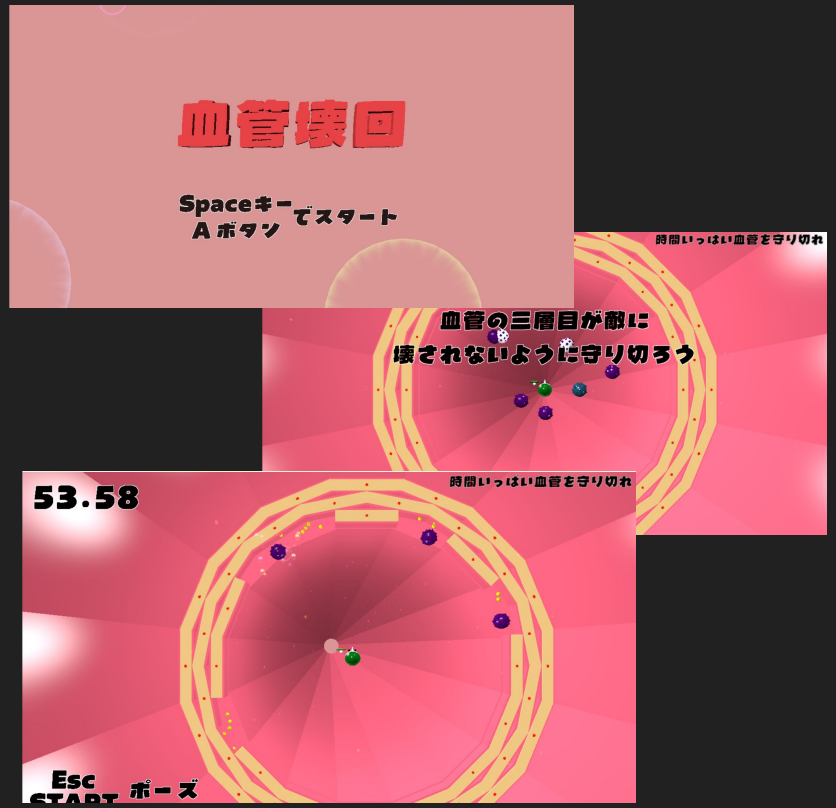
C++20 / Visual Studio 2026 / DirectX 12(描画) / XAudio2(サウンド) / Windows SDK 10.0.26100

外部ライブラリ

Assimp（3Dモデル読み込み）, DirectXTex（テクスチャ・画像読み込み）, Dear ImGui（リアルタイムデバッグUI）

【 担当箇所 】

- 自作エンジン開発・移植（DirectX 12基盤開発、アプリケーションの自作エンジンへの統合）
- シーン管理基盤（Title、InGame、Clear、GameOverの遷移制御）
- 当たり判定・ロジック（プレイヤーと敵、壁との厳密なコリジョン処理）
- 特殊演出・エフェクト（血流・血管表現の3Dパーティクル実装）



自作のDirectX 12エンジンを使用して制作した、見下ろし型の疑似3Dアクションゲームです。「血管の中」を舞台にし、チューブ状のマップや血流パーティクルで独自の世界観を表現しました。ゲームロジックはXY平面ベースの2Dアクションとして構築しつつ、描画には3Dモデルを用いることで、軽快な操作性とリッチなビジュアルを両立させています。

モノクション

ジャンル：2Dパズルアクション

制作時期：2024年10月～11月頃（約1ヶ月）

開発人数：3人（プログラマーのみ）

開発環境・API

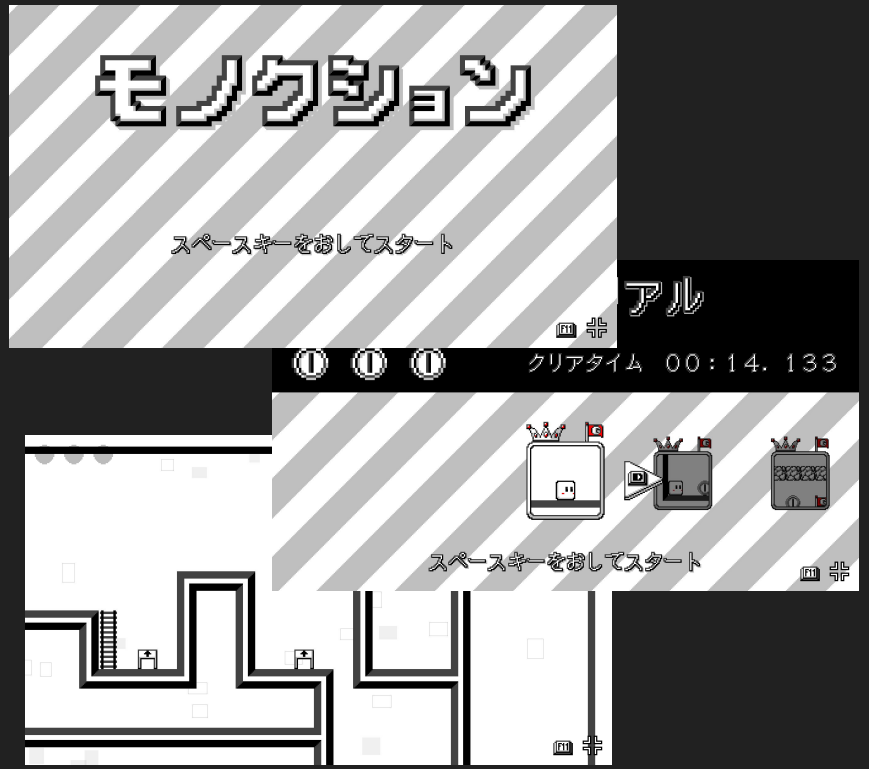
C++ / Visual Studio 2022 / 学内2D描画ライブラリ(KamataEngne Novice)

外部ライブラリ

なし

【 担当箇所 】

- プレイヤーアクション実装（鎖を使ったターザンギミックの物理挙動・関数実装）
- レベルデザイン・マップ制作（ステージエディタを活用したギミック配置）
- UI・画像制作（各種アイコン素材の作成）



1年次に3人チームで制作した、強制的に前進し続けるプレイヤーをゴールまで導く 2Dパズルアクションゲーム。一瞬の判断で正しいルートを選んだり、ギミックを駆使して進むスリリングなゲーム性が特徴です。自身は、鎖（ワイヤー）にぶら下がるターザン挙動などのアクション実装と、プレイヤーを悩ませるステージ設計（レベルデザイン）を担当しました。

累計個人5回、チーム9回のゲーム制作を経験

















